

TASKNORM: Rethinking Batch Normalization for Meta-Learning

John Bronskill^{*1} Jonathan Gordon^{*1} James Requeima¹² Sebastian Nowozin³ Richard E. Turner¹³

Abstract

Modern meta-learning approaches for image classification rely on increasingly deep networks to achieve state-of-the-art performance, making batch normalization an essential component of meta-learning pipelines. However, the hierarchical nature of the meta-learning setting presents several challenges that can render conventional batch normalization ineffective, giving rise to the need to rethink normalization in this setting. We evaluate a range of approaches to batch normalization for meta-learning scenarios, and develop a novel approach that we call **TASKNORM**. Experiments on fourteen datasets demonstrate that the choice of batch normalization has a dramatic effect on both classification accuracy and training time for both gradient based- and gradient-free meta-learning approaches. Importantly, TASKNORM is found to consistently improve performance. Finally, we provide a set of best practices for normalization that will allow fair comparison of meta-learning algorithms.

1. Introduction

Meta-learning, or learning to learn (Thrun & Pratt, 2012; Schmidhuber, 1987), is an appealing approach for designing learning systems. It enables practitioners to construct models and training procedures that explicitly target desirable characteristics such as sample-efficiency and out-of-distribution generalization. Meta-learning systems have been demonstrated to excel at complex learning tasks such as few-shot learning (Snell et al., 2017; Finn et al., 2017) and continual learning (Nagabandi et al., 2019; Requeima et al., 2019a; Jerfel et al., 2019).

Recent approaches to meta-learning rely on increasingly deep neural network based architectures to achieve state-of-

the-art performance in a range of benchmark tasks (Finn et al., 2017; Mishra et al., 2018; Triantafillou et al., 2020; Requeima et al., 2019a). When constructing very deep networks, a standard component is the use of normalization layers (NL). In particular, in the image-classification domain, batch normalization (BN; Ioffe, 2017) is crucial to the successful training of very deep convolutional networks.

However, as we discuss in Section 3, standard assumptions of the meta-learning scenario violate the assumptions of BN and vice-versa, complicating the deployment of BN in meta-learning. Many papers proposing novel meta-learning approaches employ different forms of BN for the proposed models, and some forms make implicit assumptions that, while improving benchmark performance, may result in potentially undesirable behaviours. Moreover, as we demonstrate in Section 5, performance of the trained models can vary significantly based on the form of BN employed, confounding comparisons across methods. Further, naive adoption of BN for meta-learning does not reflect the statistical structure of the data-distribution in this scenario. In contrast, we propose a novel variant of BN – TASKNORM – that explicitly accounts for the statistical structure of the data distribution. We demonstrate that by doing so, TASKNORM further accelerates training of models using meta-learning while achieving improved test-time performance. Our main contributions are as follows:

- We identify and highlight several issues with BN schemes used in the recent meta-learning literature.
- We propose TASKNORM, a novel variant of BN which is tailored for the meta-learning setting.
- In experiments with fourteen datasets, we demonstrate that TASKNORM consistently outperforms competing methods, while making less restrictive assumptions than its strongest competitor.

2. Background and Related Work

In this section we lay the necessary groundwork for our investigation of batch normalization in the meta-learning scenario. Our focus in this work is on image classification. We denote images $x \in \mathbb{R}^{C \times W \times H}$ where W is the image width, H the image height, C the number of image channels. Each image is associated with a label $y \in \{1, \dots, M\}$

^{*}Equal contribution ¹University of Cambridge ²Invenia Labs ³Microsoft Research. Correspondence to: John Bronskill <jfb54@cam.ac.uk>.

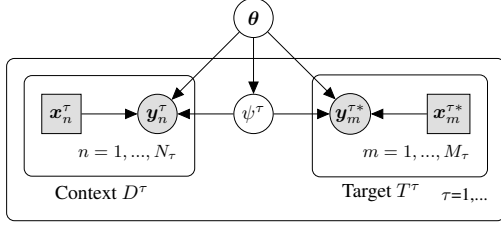


Figure 1. Directed graphical model for multi-task meta-learning.

where M is the number of image classes. Finally, a dataset is denoted $D = \{(x_n, y_n)\}_{n=1}^N$.

2.1. Meta-Learning

We consider the meta-learning classification scenario. Rather than a single, large dataset D , we assume access to a dataset $\mathcal{D} = \{\tau_t\}_{t=1}^K$ comprising a large number of training tasks τ_t , drawn i.i.d. from a distribution $p(\tau)$. The data for a task τ consists of a context set $D^\tau = \{(x_n^\tau, y_n^\tau)\}_{n=1}^{N_\tau}$ with N_τ elements with the inputs x_n^τ and labels y_n^τ observed, and a target set $T^\tau = \{(x_m^{\tau*}, y_m^{\tau*})\}_{m=1}^{M_\tau}$ with M_τ elements for which we wish to make predictions. Here the inputs $x^{\tau*}$ are observed and the labels $y^{\tau*}$ are only observed during meta-training (i.e., training of the meta-learning algorithm). The examples from a single task are assumed i.i.d., but examples across tasks are not. Note that the target set examples are drawn from the same set of labels as the examples in the context set.

At meta-test time, the meta-learner is required to make predictions for target set inputs of unseen tasks. Often, the assumption is that test tasks will include classes that have not been seen during meta-training, and D^τ will contain only a few observations. The goal of the meta-learner is to process D^τ , and produce a model that can make predictions for any test inputs $x^{\tau*} \in T^{\tau*}$ associated with the task.

Meta-Learning as Hierarchical Probabilistic Modelling

A general and useful view of meta-learning is through the perspective of hierarchical probabilistic modelling (Heskes, 2000; Bakker & Heskes, 2003; Grant et al., 2018; Gordon et al., 2019). A standard graphical representation of this modelling approach is presented in Figure 1. Global parameters θ encode information shared across all tasks, while local parameters ψ^τ encode information specific to task τ . This model introduces a hierarchy of latent parameters, corresponding to the hierarchical nature of the data distribution.

A general approach to meta-learning is to design inference procedures for the task-specific parameters $\psi^\tau = f_\phi(D^\tau)$ conditioned on the context set (Grant et al., 2018; Gordon et al., 2019), where f is parameterized by additional parameters ϕ . Thus, a meta-learning algorithm defines a predictive distribution parameterized by θ and ϕ as

$p(y_m^{\tau*} | x_m^{\tau*}, f_\phi(D^\tau), \theta)$. This perspective relates to the inner and outer loops of meta-learning algorithms (Grant et al., 2018; Rajeswaran et al., 2019): the inner loop uses f_ϕ to provide local updates to ψ , while the outer loop provides predictions for target points. Below, we use this view to summarize a range of meta-learning approaches.

Episodic Training The majority of modern meta-learning methods employ episodic training (Vinyals et al., 2016). During meta-training, a task τ is drawn from $p(\tau)$ and randomly split into a context set D^τ and target set T^τ . The meta-learning algorithm’s inner-loop is then applied to the context set to produce ψ^τ . With θ and ψ^τ , the algorithm can produce predictions for the target set inputs $x_m^{\tau*}$.

Given a differentiable loss function, and assuming that f_ϕ is also differentiable, the meta-learning algorithm can then be trained with stochastic gradient descent algorithms. Using log-likelihood as an example loss function, we may express a meta-learning objective for θ and ϕ as

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{p(\tau)} \left[\sum_{m=1}^{M_\tau} \log p(y_m^{\tau*} | x_m^{\tau*}, f_\phi(D^\tau), \theta) \right]. \quad (1)$$

Common Meta-Learning Algorithms There has been an explosion of meta-learning algorithms proposed in recent years. For an in-depth review see Hospedales et al. (2020). Here, we briefly introduce several methods, focusing on those that are relevant to our experiments. Arguably the most widely used is the gradient-based approach, the canonical example for modern systems being MAML (Finn et al., 2017). MAML sets θ to be the initialization of the neural network parameters. The local parameters ψ^τ are the network parameters after applying one or more gradient updates based on D^τ . Thus, f in the case of MAML is a gradient-based procedure, which may or may not have additional parameters (e.g., learning rate).

Another widely used class of meta-learners are amortized-inference based approaches e.g. VERSA (Gordon et al., 2019) and CNAPS (Requeima et al., 2019a). In these methods, θ parameterizes a shared feature extractor, and ψ a set of parameters used to adapt the network to the local task, which include a linear classifier and possibly additional parameters of the network. For these models, f is implemented via hyper-networks (Ha et al., 2016) with parameters ϕ . An important special case of this approach is Prototypical Networks (ProtoNets) (Snell et al., 2017), which replace ψ with nearest neighborhood classification in the embedding space of a learned feature extractor g_θ .

2.2. Normalization Layers in Deep Learning

Normalization layers (NL) for deep neural networks were introduced by Ioffe & Szegedy (2015) to accelerate the train-

ing of neural networks by allowing the use of higher learning rates and decreasing the sensitivity to network initialization. Since their introduction, they have proven to be crucial components in the successful training of ever-deeper neural architectures. Our focus is the few-shot image classification setting, and as such we concentrate on NLs for 2D convolutional networks. The input to a NL is $A = (a_1, \dots, a_B)$, a batch of B image-shaped activations or pre-activations, to which the NL is applied as

$$a'_n = \gamma \left(\frac{a_n - \mu}{\sqrt{\sigma^2 + \epsilon}} \right) + \beta, \quad (2)$$

where μ and σ are the normalization moments, γ and β are learned parameters, ϵ is a small scalar to prevent division by 0, and operations between vectors are element-wise. NLs differ primarily by how the normalization moments are computed. The first such layer – batch normalization (BN) – was introduced by Ioffe & Szegedy (2015). A BN layer distinguishes between training and test modes. At training time, BN computes the moments as

$$\mu_{BN_c} = \frac{1}{BHW} \sum_{b=1}^B \sum_{w=1}^W \sum_{h=1}^H a_{bwhc}, \quad (3)$$

$$\sigma_{BN_c}^2 = \frac{1}{BHW} \sum_{b=1}^B \sum_{w=1}^W \sum_{h=1}^H (a_{bwhc} - \mu_{BN_c})^2. \quad (4)$$

Here, $\mu_{BN}, \sigma_{BN}^2, \gamma, \beta \in \mathbb{R}^C$. As μ_{BN} and σ_{BN}^2 depend on the batch of observations, BN can be susceptible to failures if the batches at test time differ significantly from training batches, e.g., for streaming predictions. To counteract this, at training time, a running mean and variance, $\mu_r, \sigma_r \in \mathbb{R}^C$, are also computed for each BN layer over all training tasks and stored. At test time, test activations a are normalized using Equation (2) with the statistics μ_r and σ_r in place of the batch statistics. Importantly, BN relies on the implicit assumption that D comprises i.i.d. samples from some underlying distribution.

More recently, additional NLs have been introduced. Many of these methods differ from standard BN in that they normalize each instance independently of the remaining instances in the batch, making them more resilient to batch distribution shifts at test time. These include instance normalization (Ulyanov et al., 2016), layer normalization (Ba et al., 2016), and group normalization (Wu & He, 2018). These are discussed further in Section 3.3.

2.3. Desiderata for Meta-Learning Normalization Layers

As modern approaches to meta-learning systems routinely employ deep networks, NLs become essential for efficient training and optimal classification performance. For BN in the standard supervised settings, i.i.d. assumptions about the

data distribution imply that estimating moments from the training set will provide appropriate normalization statistics for test data. However, this does not hold in the meta-learning scenario, for which data points are only assumed to be i.i.d. within a specific task. Therefore, the choice of what moments to use when applying a NL to the context and target set data points, during both meta-training and meta-test time, is key.

As a result, recent meta-learning approaches employ several normalization procedures that differ according to these design choices. A range of choices are summarized in Figure 2. As we discuss in Section 3 and demonstrate with experimental results, some of these have implicit, undesirable assumptions which have significant impact on both predictive performance and training efficiency. We argue that an appropriate NL for the meta-learning scenario requires consideration of the data-generating assumptions associated with the setting. In particular, we propose the following desiderata for a NL when used for meta-learning:

1. Improves speed and stability of training without harming test performance (test set accuracy or log-likelihood);
2. Works well across a range of context set sizes;
3. Is non-transductive, thus supporting inference at meta-test time in a variety of circumstances.

A non-transductive meta-learning system makes predictions for a single test set label conditioned only on a single input and the context set, while a transductive meta-learning system conditions on additional samples from the test set:

$$\underbrace{p(y_i^{\tau*} | x_i^{\tau*}, D^{\tau})}_{\text{non-transductive}} ; \underbrace{p(y_i^{\tau*} | x_{i=1:m}^{\tau*}, D^{\tau})}_{\text{transductive}}. \quad (5)$$

We argue that there are two key issues with transductive meta-learners. The first is that transductive learning is sensitive to the distribution over the target set used during meta-training, and as such is less generally applicable than non-transductive learning. For example, transductive learners may fail to make good predictions if target sets contain a different class balance than what was observed during meta-training, or if they are required to make predictions for one example at a time. Transductive learners can also violate privacy constraints. In Table 1 and Appendix D, we provide empirical demonstrations of these failure cases.

The second issue is that transductive learners have more information available than non-transductive learners at prediction time, which may lead to unfair comparisons. It is worth noting that some meta-learning algorithms are specifically designed to leverage transductive inference (e.g., Ren et al., 2018; Liu et al., 2019), though we do not discuss them in this work. In Section 5 we demonstrate that there are significant performance differences for a model when trained transductively versus non-transductively.

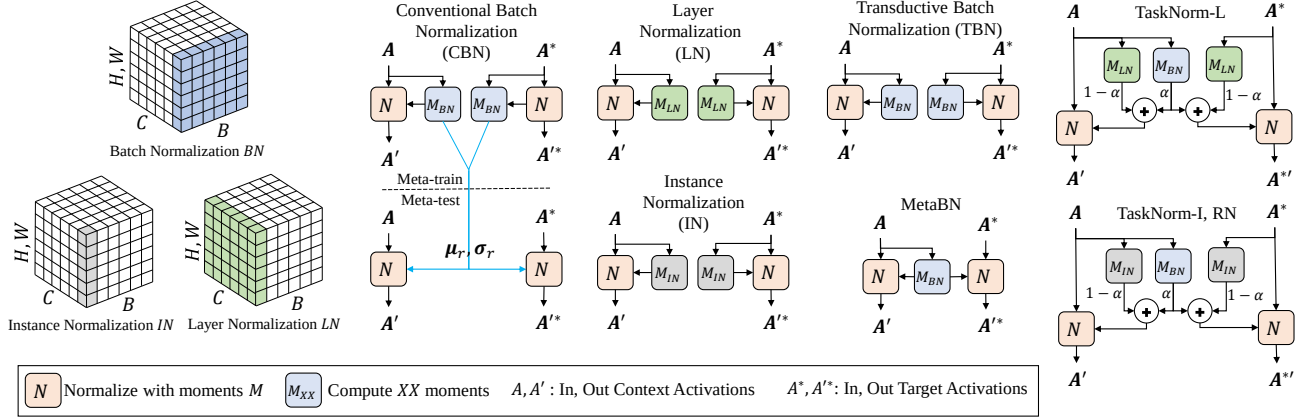


Figure 2. A range of options for batch normalization for meta-learning. The cubes on the left depict the dimensions over which different moments are calculated for normalization of 2D convolutional layers. The computational diagrams on the right show how context and target activations are processed for various normalization methods. For all methods except conventional BN (CBN), the processing is identical at meta-train and meta-test time. Cube diagrams are derived from Wu & He (2018).

3. Normalization Layers for Meta-learning

In this section, we discuss several normalization schemes that can and have been applied in the recent meta-learning literature, highlighting the modelling assumptions and effects of different design choices. Throughout, we assume that the meta-learning algorithm is constructed such that the context-set inputs are passed through every neural-network module that the target set inputs are passed through at prediction time. This implies that moments are readily available from both the context and target set observations for any normalization layer, and is the case for many widely-used meta-learning models (e.g., Finn et al., 2017; Snell et al., 2017; Gordon et al., 2019).

To illustrate our arguments, we provide experiments with MAML running simple, but widely used few-shot learning tasks from the Omniglot (Lake et al., 2011) and minilma-genet (Ravi & Larochelle, 2017) datasets. The results of these experiments are provided in Table 1, and full experimental details in Appendix B.

3.1. Conventional Usage of Batch Normalization (CBN)

We refer to *conventional* batch normalization (CBN) as that defined by Ioffe & Szegedy (2015) and as outlined in Section 2.2. In the context of meta-learning, this involves normalizing tasks with computed moments at meta-train time, and using the accumulated running moments to normalize the tasks at meta-test time (see CBN in Figure 2).

We highlight two important issues with the use of CBN for meta-learning. The first is that, from the graphical model perspective, this is equivalent to lumping μ and σ with the global parameters θ , i.e., they are learned from the meta-training set and shared across all tasks at meta-test time.

We might expect CBN to perform poorly in meta-learning applications since the running moments are global across all tasks while the task data is only i.i.d. locally within a task, i.e., CBN does not satisfy desiderata 1. This is corroborated by our results (Table 1), where we demonstrate that using CBN with MAML results in very poor predictive performance - no better than chance. The second issue is that, as demonstrated by Wu & He (2018), using small batch sizes leads to inaccurate moments, resulting in significant increases in model error. Importantly, the small batch setting may occur often in meta-learning, for example in the 1-shot scenario. Thus, CBN does not satisfy desiderata 2.

Despite these issues, CBN is sometimes used, e.g., by Snell et al. (2017), though testing was performed only on Omniglot and miniImagenet where the distribution of tasks is homogeneous (Triantafillou et al., 2020). In Section 5, we show that Batch renormalization (BRN; Ioffe, 2017) can exhibit poor predictive performance in meta-learning scenarios (see Appendix A.1 for further details).

3.2. Transductive Batch Normalization (TBN)

Another approach is to do away with the running moments used for normalization at meta-test time, and replace these with context / target set statistics. Here, context / target set statistics are used for normalization, both at meta-train and meta-test time. This is the approach taken by the authors of MAML (Finn et al., 2017), and, as demonstrated in our experiments, seems to be crucial to achieve the reported performance. From the graphical model perspective, this implies associating the normalization statistics with neither θ nor ψ , but rather with a special set of parameters that is

¹See for example (Finn, 2017) for a reference implementation.

local for each set (i.e., normalization statistics for T^τ are independent of D^τ). We refer to this approach as *transductive* batch normalization (TBN; see Figure 2).

Unsurprisingly, Nichol et al. (2018) found that using TBN provides a significant performance boost in all cases they tested, which is corroborated by our results in Table 1. In other words, TBN achieves desiderata 2, and, as we demonstrate in Section 5, desiderata 1 as well. However, it is transductive. Due to the ubiquity of MAML, many competitive meta-learning methods (e.g. Gordon et al., 2019) have adopted TBN. However, in the case of TBN, transductivity is rarely stated as an explicit assumption, and may often confound the comparison among methods (Nichol et al., 2018). Importantly, we argue that to ensure comparisons in experimental papers are rigorous, meta-learning methods that are transductive must be labeled as such.

3.3. Instance-Based Normalization Schemes

An additional class of non-transductive NLs are *instance-based* NLs. Here, both at meta-train and meta-test time, moments are computed separately for each instance, and do not depend on other observations. From a modelling perspective, this corresponds to treating μ and σ as local at the *observation* level. As instance-based NLs do not depend on the context set size, they perform equally well across context-set sizes (desiderata 2). However, as we demonstrate in Section 5, the improvements in predictive performance are modest compared to more suitable NLs and they are worse than CBN in terms of training efficiency (thus not meeting desiderata 1). Below, we discuss two examples, with a third discussed in Appendix A.2.

Layer Normalization (LN; Ba et al., 2016) LN (see Figure 2) has been shown to improve performance compared to CBN in recurrent neural networks, but does not offer the same gains for convolutional neural networks (Ba et al., 2016). The LN moments are computed as:

$$\mu_{LN_b} = \frac{1}{HWC} \sum_{w=1}^W \sum_{h=1}^H \sum_{c=1}^C a_{bwhc}, \quad (6)$$

$$\sigma_{LN_b}^2 = \frac{1}{HWC} \sum_{w=1}^W \sum_{h=1}^H \sum_{c=1}^C (a_{bwhc} - \mu_{LN_b})^2 \quad (7)$$

where $\mu_{LN}, \sigma_{LN}^2 \in \mathbb{R}^B$. While non-transductive, Table 1 demonstrates that LN falls far short of TBN in terms of accuracy. Further, in Section 5 we demonstrate that LN lacks in training efficiency when compared to other NLs.

Instance Normalization (IN; Ulyanov et al., 2016) IN (see Figure 2) has been used in a wide variety of image

generation applications. The IN moments are computed as:

$$\mu_{IN_{bc}} = \frac{1}{HW} \sum_{w=1}^W \sum_{h=1}^H a_{bwhc}, \quad (8)$$

$$\sigma_{IN_{bc}}^2 = \frac{1}{HW} \sum_{w=1}^W \sum_{h=1}^H (a_{bwhc} - \mu_{IN_{bc}})^2 \quad (9)$$

where $\mu_{IN}, \sigma_{IN}^2 \in \mathbb{R}^{B \times C}$. Table 1 demonstrates that IN has superior predictive performance to that of LN, but falls considerably short of TBN. In Section 5 we show that IN lacks in training efficiency when compared to other NLs.

4. Task Normalization

In the previous section, we demonstrated that it is not immediately obvious how NLs should be designed for meta-learning applications. We now develop TASKNORM, the first NL that is specifically tailored towards this scenario. TASKNORM is motivated by the view of meta-learning as hierarchical probabilistic modelling, discussed in Section 2.1. Given this hierarchical view of the model parameters, the question that arises is, how should we treat the normalization statistics μ and σ ? Figure 1 implies that the data associated with a task τ are i.i.d. *only when conditioning on both θ and ψ^τ* . Thus, the normalization statistics μ and σ should be local at the task level, i.e., absorbed into ψ^τ . Further, the view that ψ^τ should be inferred conditioned on D^τ implies that the normalization statistics for the target set should be computed directly from the context set. Finally, our desire for a non-transductive scheme implies that any contribution from points in the target should not affect the normalization for other points in the target set, i.e., when computing μ and σ for a particular observation $x^{\tau*} \in T^\tau$, the NL should only have access to D^τ and $x^{\tau*}$.

4.1. Meta-Batch Normalization (METABN)

This perspective leads to our definition of METABN, which is a simple adaptation of CBN for the meta-learning setting. In METABN, the context set alone is used to compute the normalization statistics for *both* the context and target sets, both at meta-train and meta-test time (see Figure 2). To our knowledge, METABN has not been described in any publication, but concurrent to this work, it is used in the implementation of Meta-Dataset (Triantafillou et al., 2019).

METABN meets almost all of our desiderata, it (i) is non-transductive since the normalization of a test input does not depend on other test inputs in the target set, and (ii) as we demonstrate in Section 5, it improves training speed while maintaining accuracy levels of meta-learning models. However, as we demonstrate in Section 5, METABN performs less well for small context sets. This is because moment estimates will have high-variance when there is little data,

and is similar to the difficulty of using BN with small-batch training (Wu & He, 2018). To address this issue, we introduce the following extension to METABN, which yields our proposed normalization scheme – TASKNORM.

4.2. TASKNORM

The key intuition behind TASKNORM is to normalize a task with the context set moments in combination with a set of non-transductive, secondary moments computed from the input being normalized. A blending factor α between the two sets of moments is learned during meta-training. The motivation for TASKNORM is as follows: when the context set D^τ is small (e.g. 1-shot or few-shot learning) the context set alone will lead to noisy and inaccurate estimates of the “true” task statistics. In such cases, a secondary set of moments may improve the estimate of the moments, leading to better training efficiency and predictive performance in the low data regime. Further, this provides information regarding $\mathbf{x}^{\tau*}$ at prediction time while maintaining non-transductivity. The pooled moments for TASKNORM are computed as:

$$\boldsymbol{\mu}_{TN} = \alpha \boldsymbol{\mu}_{BN} + (1 - \alpha) \boldsymbol{\mu}_+, \quad (10)$$

$$\sigma_{TN}^2 = \alpha (\sigma_{BN}^2 + (\boldsymbol{\mu}_{BN} - \boldsymbol{\mu}_{TN})^2) + (1 - \alpha) (\sigma_+^2 + (\boldsymbol{\mu}_+ - \boldsymbol{\mu}_{TN})^2), \quad (11)$$

where $\boldsymbol{\mu}_{TN}, \sigma_{TN} \in \mathbb{R}^{B \times C}$, $\boldsymbol{\mu}_+, \sigma_+^2$ are additional moments from a non-transductive NL such as LN or IN computed using activations from the example being normalized (see Figure 2), and $\boldsymbol{\mu}_{BN}$ and σ_{BN} are computed from D^τ . Equation (11) is the standard *pooled variance* when combining the variance of two Gaussian estimators.

Importantly, we parameterize $\alpha = \text{SIGMOID}(\text{SCALE}|D^\tau| + \text{OFFSET})$, where the SIGMOID function ensures that $0 \leq \alpha \leq 1$, and the scalars SCALE and OFFSET are learned during meta-training. This enables us to learn how much each set should contribute to the estimate of task statistics as a function of the context-set size $|D^\tau|$. Figure 3a depicts the value of α as a function of context set size $|D^\tau|$ for a representative set of trained TASKNORM layers. In general, when the context size is suitably large ($N_\tau > 25$), α is close to unity, i.e., normalization is carried out entirely with the context set in those layers. When the context size is smaller, there is a mix of the two sets of moments.

Allowing each TASKNORM layer to separately adapt to the size of the context set (as opposed to learning a fixed α per layer) is crucial in the meta-learning setting, where we expect the size of D^τ to vary, and are often particularly interested in the “few-shot” regime. Figure 3b plots the line $\text{SCALE}|D^\tau| + \text{OFFSET}$ for same set of NLs as Figure 3a. The algorithm has learned that the SCALE parameter is non-zero and the OFFSET is almost zero in all cases indicating

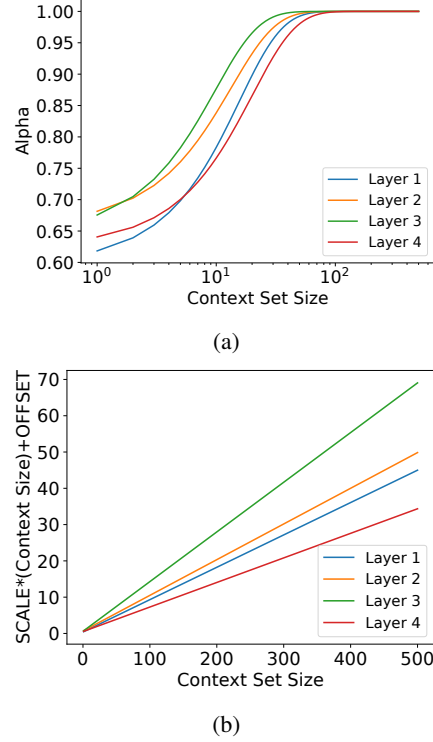


Figure 3. Plots of: (a) α versus context set size, and (b) α versus $\text{SCALE}|D^\tau| + \text{OFFSET}$ for the first NL in each of the four layers in the feature extractor for the TASKNORM-I model.

the importance of having α being a function of context size. In Appendix E, we provide an ablation study demonstrating the importance of our proposed parameterization of α . If the context size is fixed, we do not use the full parameterization, but learn a single value for alpha directly. The computational cost of TASKNORM is marginally greater than CBN’s. As a result, per-iteration time increases only slightly. However, as we show in Section 5, TASKNORM converges faster than CBN.

In related work, Nam & Kim (2018) define Batch-Instance Normalization (BIN) that combines the results of CBN and IN with a learned blending factor in order to attenuate unnecessary styles from images. However, BIN blends the output of the individual CBN and IN normalization operations as opposed to blending the moments. Finally, we note that Reptile (Nichol et al., 2018) uses a non-transductive form of task normalization that involves normalizing examples from the target set one example at a time with the moments of the context set augmented with the single example. We refer to this approach as *reptile* normalization or RN. It is easy to show that RN is a special case of TASKNORM augmented with IN when $\alpha = |D^\tau|/(1 + |D^\tau|)$. In Section 5, we show that reptile normalization falls short of TASKNORM, supporting the intuition that learning the value of α is preferable to fixing a value.

5. Experiments

In this section, we evaluate TASKNORM along with a range of competitive normalization approaches.² The goal of the experiments is to evaluate the following hypotheses: (i) Meta-learning algorithms are sensitive to the choice of NL; (ii) TBN will, in general, outperform non-transductive NLs; and (iii) NLs that consider the meta-learning data assumptions (TASKNORM, METABN, RN) will outperform ones that do not (CBN, BRN, IN, LN, etc.).

5.1. Small Scale Few-Shot Classification Experiments

We evaluate TASKNORM and a set of NLs using the first order MAML and ProtoNets algorithms on the Omniglot and miniImageNet datasets under various way (the number of classes used in each task) and shot (the number of context set examples used per class) configurations. This setting is smaller scale, and considers only fixed-sized context and target sets. Configuration and training details can be found in Appendix B.

Accuracy Table 1 and Table C.1 show accuracy results for various normalization methods on the Omniglot and miniImageNet datasets using the first order MAML and the ProtoNets algorithms, respectively. We compute the average rank in an identical manner to Triantafillou et al. (2020).

For MAML, TBN is clearly the best method in terms of classification accuracy. The best non-transductive approach is TASKNORM that uses IN augmentation (TASKNORM-I). The two methods using instance-based normalization (LN, IN) do significantly less well than methods designed with meta-learning desiderata in mind (i.e. TASKNORM, MetaBN, and RN). The methods using running averages at meta-test time (CBN, BRN) fare the worst. Figure 4a compares the performance of MAML on unseen tasks from miniImageNet when trained with TBN, IN, METABN, and TASKNORM, as a function of the number of shots per class in D^τ , and demonstrates that these trends are consistent across the low-shot range.

Note that when meta-testing occurs one example at a time (e.g. in the streaming data scenario) or one class at a time (unbalanced class distribution scenario), accuracy for TBN drops dramatically compared to the case where all the examples are tested at once. This is an important drawback of the transductive approach. All of the other NLs in the table are non-transductive and do not suffer a decrease in accuracy when tested an example at a time or a class at a time.

Compared to MAML, the ProtoNets algorithm is much less sensitive to the NL used. Table C.1 indicates that with the exception of IN, all of the normalization methods yield

good performance. We suspect that this is due to the fact that in ProtoNets employs a parameter-less nearest neighbor classifier and no gradient steps are taken at meta-test time, reducing the importance of normalization. The top performer is LN which narrowly edges out TaskNorm-L and CBN. Interestingly, TBN is not on top and TASKNORM-I lags as IN is the least effective method.

Training Speed Figure 4b plots validation accuracy versus training iteration for the first order MAML algorithm training on Omniglot 5-way-5-shot. TBN is the most efficient in terms of training convergence. The best non-transductive method is again TASKNORM-I, which is only marginally worse than TBN and just slightly better than TASKNORM-L. Importantly, TASKNORM-I is superior to either of MetaBN and IN alone in terms of training efficiency. Figure C.1a depicts the training curves for the ProtoNets algorithm. With the exception of IN which converges to a lower validation accuracy, all NLs converge at the the same speed.

For the MAML algorithm, the experimental results support our hypotheses. Performance varies significantly across NLs. TBN outperformed all methods in terms of classification accuracy and training efficiency, and TASKNORM is the best non-transductive approach. Finally, The meta-learning specific methods outperformed the more general ones. The picture for ProtoNets is rather different. There is little variability across NLs, TBN lagged the most consistent method LN in terms of accuracy, and the NLs that considered meta-learning needs were not necessarily superior to those that did not.

5.2. Large Scale Few-Shot Classification Experiments

Next, we evaluate NLs on a demanding few-shot classification challenge called Meta-Dataset, composed of thirteen (eight train, five test) image classification datasets (Triantafillou et al., 2020). Experiments are carried out with CNAPS, which achieves state-of-the-art performance on Meta-Dataset (Requeima et al., 2019a) and ProtoNets. The challenge constructs few-shot learning tasks by drawing from the following distribution. First, one of the datasets is sampled uniformly; second, the “way” and “shot” are sampled randomly according to a fixed procedure; third, the classes and context / target instances are sampled. As a result, the context size D^τ will vary in the range between 5 and 500 for each task. In the meta-test phase, the identity of the original dataset is not revealed and tasks must be treated independently (i.e. no information can be transferred between them). The meta-training set comprises a disjoint and dissimilar set of classes from those used for meta-test. Details provided in Appendix B and Triantafillou et al. (2020).

²Source code is available at <https://github.com/cambridge-mlg/cnaps>

Table 1. Accuracy results for different few-shot settings on Omniglot and miniImageNet using the MAML algorithm. All figures are percentages and the $\pm$ sign indicates the 95% confidence interval. Bold indicates the highest scores. The numbers after the configuration name indicate the way and shots, respectively. The vertical lines enclose the transductive results. The *TBN*, *examples*, and *class* columns indicate accuracy when tested with all target examples at once, one example at a time, and one class at a time, respectively. All other NLs are non-transductive and yield the same result when tested by example or class.

Configuration	TBN	example	class	CBN	BRN	LN	IN	RN	MetaBN	TaskNorm-L	TaskNorm-I
Omniglot-5-1	98.4$\pm$0.7	21.6 $\pm$ 1.3	21.6 $\pm$ 1.3	20.1 $\pm$ 0.0	20.0 $\pm$ 0.0	83.0 $\pm$ 1.3	87.4 $\pm$ 1.2	92.6 $\pm$ 0.9	91.8 $\pm$ 0.9	94.0 $\pm$ 0.8	94.4 $\pm$ 0.8
Omniglot-5-5	99.2$\pm$0.2	22.0 $\pm$ 0.5	23.2 $\pm$ 0.5	20.0 $\pm$ 0.0	20.0 $\pm$ 0.0	91.0 $\pm$ 0.8	93.9 $\pm$ 0.5	98.2 $\pm$ 0.2	98.1 $\pm$ 0.3	98.0 $\pm$ 0.3	98.6 $\pm$ 0.2
Omniglot-20-1	90.9$\pm$0.5	3.7 $\pm$ 0.2	3.7 $\pm$ 0.2	5.0 $\pm$ 0.0	5.0 $\pm$ 0.0	78.1 $\pm$ 0.7	80.4 $\pm$ 0.7	89.0 $\pm$ 0.6	89.6 $\pm$ 0.5	89.6 $\pm$ 0.5	90.0$\pm$0.5
Omniglot-20-5	96.6$\pm$0.2	5.5 $\pm$ 0.2	14.5 $\pm$ 0.3	5.0 $\pm$ 0.0	5.0 $\pm$ 0.0	92.3 $\pm$ 0.2	92.9 $\pm$ 0.2	96.8$\pm$0.2	96.4 $\pm$ 0.2	96.4 $\pm$ 0.2	96.3 $\pm$ 0.2
miniImageNet-5-1	45.5$\pm$1.8	26.9 $\pm$ 1.5	26.9 $\pm$ 1.5	20.1 $\pm$ 0.0	20.4 $\pm$ 0.4	41.2 $\pm$ 1.6	40.7 $\pm$ 1.7	40.7 $\pm$ 1.7	41.6 $\pm$ 1.6	42.0 $\pm$ 1.7	42.4 $\pm$ 1.7
miniImageNet-5-5	59.7$\pm$0.9	30.3 $\pm$ 0.7	27.2 $\pm$ 0.6	20.2 $\pm$ 0.2	20.7 $\pm$ 0.5	52.8 $\pm$ 0.9	54.3 $\pm$ 0.9	57.6 $\pm$ 0.9	58.6$\pm$0.9	58.1 $\pm$ 0.9	58.7$\pm$0.9
Average Rank	1.25	-	-	8.42	8.58	6.58	5.75	4.00	3.67	3.75	3.00

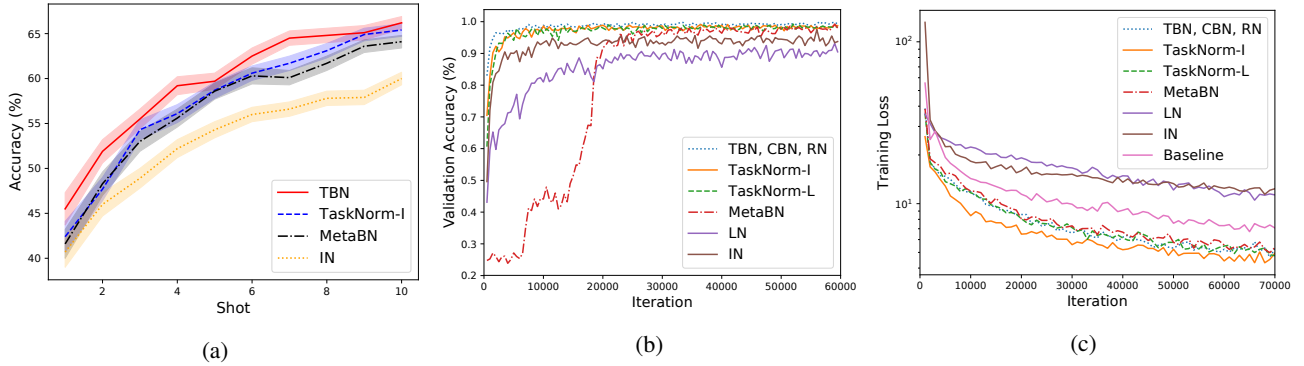


Figure 4. (a) Accuracy vs shot for MAML on 5-way miniImageNet classification. (b) Plot of validation accuracy versus training iteration using MAML for Omniglot 5-way, 5-shot corresponding to the results in Table 1. (c) Training Loss versus iteration corresponding to the results using the CNAPS algorithm in Table 2. Note that TBN, CBN, and RN all share the same meta-training step.

Accuracy The classification accuracy results for CNAPS and ProtoNets on Meta-Dataset are shown in Table 2 and Table 3, respectively. In the case of ProtoNets, all the NLs specifically designed for meta-learning scenarios outperform TBN in terms of classification accuracy based on their average rank over all the datasets. For CNAPS, both RN and TASKNORM-I meet or exceed the rank of TBN. This may be as $|D^T|$ (i) is quite large in Meta-Dataset, and (ii) may be imbalanced w.r.t. classes, making prediction harder with transductive NLs. TASKNORM-I comes out as the clear winner ranking first in 11 and 10 of the 13 datasets using CNAPS and ProtoNets, respectively. This supports the hypothesis that augmenting the BN moments with a second, instance based set of moments and learning the blending factor α as a function of context set size is superior to fixing α to a constant value (as is the case with RN). With both algorithms, the instance based NLs fall short of the meta-learning specific ones. However, in the case of CNAPS, they outperform the running average based methods (CBN, BRN), which perform poorly. In the case of ProtoNets, BRN outperforms the instance based methods, and IN fairs the worst of all. In general, ProtoNets is less sensitive to the NL used when compared to CNAPS.

The BASELINE column in Table 2 is taken from Requeima et al. (2019a), where the method reported state-of-the-art results on Meta-Dataset. The BASELINE algorithm uses the running moments learned during pre-training of its feature extractor for normalization. Using meta-learning specific NLs (in particular TASKNORM) achieves significantly improved accuracy compared to BASELINE.

As an ablation, we have also added an additional variant of TASKNORM that blends the batch moments from the context set with the running moments accumulated during meta-training that we call TASKNORM-r. TASKNORM-r makes use of the global running moments to augment the local context statistic and it did not perform as well as the TASKNORM variants that employed local moments (i.e. TASKNORM-I and TASKNORM-L).

Training Speed Figure 4c plots training loss versus training iteration for the models in Table 2 that use the CNAPS algorithm. The fastest training convergence is achieved by TASKNORM-I. The instance based methods (IN, LN) are the slowest to converge. Note that TASKNORM converges within 60k iterations while BASELINE takes 110k iterations

Table 2. Few-shot classification results on META-DATASET using the CNAPs (top) and ProtoNets (bottom) algorithms. Meta-training performed on datasets above the dashed line. Datasets below the dashed line are entirely held out. All figures are percentages and the $\pm$ sign indicates the 95% confidence interval over tasks. Bold indicates the highest scores. Vertical lines in the TBN column indicate that this method is transductive. Numbers in the BASELINE column are from (Requeima et al., 2019a).

Dataset	TBN	Baseline	CBN	BRN	LN	IN	RN	MetaBN	TaskNorm-r	TaskNorm-L	TaskNorm-I
ILSVRC	50.2$\pm$1.0	51.3$\pm$1.0	24.8 $\pm$ 0.7	19.2 $\pm$ 0.7	45.5 $\pm$ 1.1	46.7 $\pm$ 1.0	49.7 $\pm$ 1.1	51.3$\pm$1.1	49.3 $\pm$ 1.0	51.2$\pm$1.1	50.6$\pm$1.1
Omniglot	91.4$\pm$0.5	88.0 $\pm$ 0.7	47.9 $\pm$ 1.4	60.0 $\pm$ 1.6	87.4 $\pm$ 0.8	79.7 $\pm$ 1.0	91.0$\pm$0.6	90.9$\pm$0.6	87.8 $\pm$ 0.7	90.6 $\pm$ 0.6	90.7$\pm$0.6
Aircraft	81.6 $\pm$ 0.6	76.8 $\pm$ 0.8	29.5 $\pm$ 0.9	56.3 $\pm$ 0.8	76.5 $\pm$ 0.8	74.7 $\pm$ 0.7	82.4 $\pm$ 0.6	83.9$\pm$0.6	81.1 $\pm$ 0.7	81.9 $\pm$ 0.6	83.8$\pm$0.6
Birds	74.5$\pm$0.8	71.4 $\pm$ 0.9	42.1 $\pm$ 1.0	32.6 $\pm$ 0.8	67.3 $\pm$ 0.9	64.9 $\pm$ 1.0	72.4 $\pm$ 0.8	73.2 $\pm$ 0.9	72.8 $\pm$ 0.9	72.4 $\pm$ 0.8	74.6$\pm$0.8
Textures	59.7 $\pm$ 0.7	62.5$\pm$0.7	37.5 $\pm$ 0.7	50.5 $\pm$ 0.6	60.1 $\pm$ 0.6	59.7 $\pm$ 0.7	58.6 $\pm$ 0.7	58.9 $\pm$ 0.8	63.2$\pm$0.8	57.2 $\pm$ 0.7	62.1 $\pm$ 0.7
Quick Draw	70.8 $\pm$ 0.8	71.9 $\pm$ 0.8	44.5 $\pm$ 1.0	56.7 $\pm$ 1.0	71.6 $\pm$ 0.8	68.2 $\pm$ 0.9	74.3$\pm$0.8	74.1$\pm$0.7	71.6 $\pm$ 0.8	74.3$\pm$0.8	74.8$\pm$0.7
Fungi	46.0 $\pm$ 1.0	46.0 $\pm$ 1.1	21.1 $\pm$ 0.8	26.1 $\pm$ 0.9	39.6 $\pm$ 1.0	37.8 $\pm$ 1.0	49.0$\pm$1.0	47.9$\pm$1.0	42.0 $\pm$ 1.1	47.1 $\pm$ 1.1	48.7$\pm$1.0
VGG Flower	86.6 $\pm$ 0.5	89.2$\pm$0.5	79.0 $\pm$ 0.7	75.7 $\pm$ 0.7	84.4 $\pm$ 0.6	82.6 $\pm$ 0.6	86.9 $\pm$ 0.6	85.9 $\pm$ 0.6	87.7 $\pm$ 0.6	87.3 $\pm$ 0.5	89.6$\pm$0.6
Traffic Signs	66.6$\pm$0.9	60.1 $\pm$ 0.9	38.3 $\pm$ 0.9	38.8 $\pm$ 1.2	57.3 $\pm$ 0.8	62.5 $\pm$ 0.8	66.6$\pm$0.8	58.9 $\pm$ 0.9	62.7 $\pm$ 0.8	62.0 $\pm$ 0.8	67.0$\pm$0.7
MSCOCO	41.3 $\pm$ 1.0	42.0$\pm$1.0	14.2 $\pm$ 0.7	19.1 $\pm$ 0.8	32.9 $\pm$ 1.0	40.8 $\pm$ 1.0	42.1$\pm$1.0	41.6 $\pm$ 1.1	40.1 $\pm$ 1.0	41.6 $\pm$ 1.0	43.4$\pm$1.0
MNIST	92.1 $\pm$ 0.4	88.6 $\pm$ 0.5	65.9 $\pm$ 0.8	82.5 $\pm$ 0.6	86.8 $\pm$ 0.5	89.8 $\pm$ 0.5	91.3 $\pm$ 0.4	92.1 $\pm$ 0.4	93.2$\pm$0.3	90.5 $\pm$ 0.4	92.3$\pm$0.4
CIFAR10	70.1$\pm$0.8	60.0 $\pm$ 0.8	26.1 $\pm$ 0.7	29.1 $\pm$ 0.6	55.8 $\pm$ 0.8	65.9 $\pm$ 0.8	69.7$\pm$0.7	69.6$\pm$0.8	66.9 $\pm$ 0.8	70.3$\pm$0.8	69.3$\pm$0.8
CIFAR100	55.6 $\pm$ 1.0	48.1 $\pm$ 1.0	16.7 $\pm$ 0.8	16.7 $\pm$ 0.7	37.9 $\pm$ 1.0	52.9 $\pm$ 1.0	55.0 $\pm$ 1.0	54.2 $\pm$ 1.1	53.0 $\pm$ 1.1	59.5$\pm$1.0	54.6 $\pm$ 1.1
Average Rank	3.92	5.58	10.69	10.31	7.96	7.54	3.77	4.04	5.38	4.42	2.38

Table 3. Few-shot classification results on META-DATASET using the Prototypical Networks algorithm. Datasets below the dashed line are entirely held out. Meta-training performed on datasets above the dashed line. All figures are percentages and the $\pm$ sign indicates the 95% confidence interval over tasks. Bold indicates the highest scores. Vertical lines in the TBN column indicate that this method is transductive.

Dataset	TBN	CBN	BRN	LN	IN	RN	MetaBN	TaskNorm-r	TaskNorm-L	TaskNorm-I
ILSVRC	44.7$\pm$1.0	43.6 $\pm$ 1.0	43.0 $\pm$ 1.0	33.9 $\pm$ 0.9	32.5 $\pm$ 0.9	45.1$\pm$1.0	44.2$\pm$1.0	42.7 $\pm$ 1.0	45.1$\pm$1.1	44.9$\pm$1.0
Omniglot	90.7$\pm$0.6	77.5 $\pm$ 1.1	89.1 $\pm$ 0.7	90.8$\pm$0.6	83.4 $\pm$ 0.8	90.8$\pm$0.6	90.4$\pm$0.6	88.6 $\pm$ 0.7	90.2$\pm$0.6	90.6$\pm$0.6
Aircraft	83.3 $\pm$ 0.6	77.0 $\pm$ 0.7	84.4$\pm$0.5	73.9 $\pm$ 0.7	75.0 $\pm$ 0.6	80.9 $\pm$ 0.6	82.3 $\pm$ 0.6	79.6 $\pm$ 0.6	81.2 $\pm$ 0.6	84.7$\pm$0.5
Birds	69.6 $\pm$ 0.9	67.5 $\pm$ 0.9	69.0 $\pm$ 0.9	54.1 $\pm$ 1.0	50.2 $\pm$ 1.0	68.6 $\pm$ 0.9	68.6 $\pm$ 0.8	64.2 $\pm$ 0.9	68.8 $\pm$ 0.9	71.0$\pm$0.9
Textures	61.2 $\pm$ 0.7	57.7 $\pm$ 0.7	58.0 $\pm$ 0.7	55.8 $\pm$ 0.7	45.3 $\pm$ 0.7	64.1 $\pm$ 0.7	60.5 $\pm$ 0.7	60.8 $\pm$ 0.7	63.4 $\pm$ 0.8	65.9$\pm$0.7
Quick Draw	75.0 $\pm$ 0.8	62.1 $\pm$ 1.0	74.3 $\pm$ 0.8	72.5 $\pm$ 0.8	70.8 $\pm$ 0.8	75.4 $\pm$ 0.7	74.2 $\pm$ 0.7	73.2 $\pm$ 0.8	75.4 $\pm$ 0.7	77.5$\pm$0.7
Fungi	46.4 $\pm$ 1.0	43.6 $\pm$ 1.0	46.5 $\pm$ 1.0	33.2 $\pm$ 1.1	29.8 $\pm$ 1.0	46.7 $\pm$ 1.0	46.5 $\pm$ 1.0	42.3 $\pm$ 1.1	46.5 $\pm$ 1.0	49.6$\pm$1.1
VGG Flower	83.1 $\pm$ 0.6	82.3 $\pm$ 0.6	84.5 $\pm$ 0.6	78.3 $\pm$ 0.8	69.4 $\pm$ 0.8	84.4 $\pm$ 0.7	86.0$\pm$0.6	81.1 $\pm$ 0.7	82.9 $\pm$ 0.7	83.2 $\pm$ 0.6
Traffic Signs	64.0$\pm$0.8	59.5 $\pm$ 0.8	65.7$\pm$0.8	69.1$\pm$0.7	60.7 $\pm$ 0.8	66.0 $\pm$ 0.8	63.2 $\pm$ 0.8	64.9 $\pm$ 0.8	67.0 $\pm$ 0.7	65.8 $\pm$ 0.7
MSCOCO	38.2 $\pm$ 1.0	36.6 $\pm$ 1.0	38.4$\pm$1.0	30.1 $\pm$ 0.9	27.7 $\pm$ 0.9	37.3 $\pm$ 1.0	38.6$\pm$1.1	35.4 $\pm$ 1.0	39.2$\pm$1.0	38.5$\pm$1.0
MNIST	93.4 $\pm$ 0.4	86.5 $\pm$ 0.6	91.9 $\pm$ 0.4	94.0$\pm$0.4	87.4 $\pm$ 0.5	93.9$\pm$0.4	93.9$\pm$0.4	92.5 $\pm$ 0.4	91.9 $\pm$ 0.4	93.3 $\pm$ 0.4
CIFAR10	64.7 $\pm$ 0.8	57.3 $\pm$ 0.8	60.1 $\pm$ 0.8	51.5 $\pm$ 0.8	50.5 $\pm$ 0.8	62.3 $\pm$ 0.8	63.0 $\pm$ 0.8	61.4 $\pm$ 0.8	66.9$\pm$0.8	67.6$\pm$0.8
CIFAR100	48.0 $\pm$ 1.1	43.1 $\pm$ 1.0	43.9 $\pm$ 1.0	34.0 $\pm$ 0.9	32.1 $\pm$ 1.0	47.2 $\pm$ 1.1	47.0 $\pm$ 1.0	45.2 $\pm$ 1.0	51.3$\pm$1.1	50.0$\pm$1.0
Average Rank	4.04	8.19	5.31	7.46	9.58	3.65	3.96	6.73	3.58	2.50

and IN takes 200k. Figure C.1b shows the training curves for the ProtoNets algorithm. The convergence speed trends are very similar to CNAPs, with TASKNORM-I the fastest.

Our results demonstrate that TASKNORM is the best approach for normalizing tasks on the large scale Meta-Dataset benchmark in terms of classification accuracy and training efficiency. Here, we see high sensitivity of performance across NLs. Interestingly, in this setting TASKNORM-I outperformed TBN in classification accuracy, as did both RN and METABN. This refutes the hypothesis that TBN will always outperform other methods due to its transductive property, and implies that designing NL methods specifically for meta-learning has significant value. In general, the meta-learning specific methods outperformed more general NLs, supporting our third hypothesis. We suspect the reason that TASKNORM outperforms other methods is due to its

ability to adaptively leverage information from both D^T and $\mathcal{X}^{\tau^*}$ when computing moments, based on the size of D^T .

6. Conclusions

We have identified and specified several issues and challenges with NLs for the meta-learning setting. We have introduced a novel variant of batch normalization – that we call TASKNORM – which is geared towards the meta-learning setting. Our experiments demonstrate that TASKNORM achieves performance gains in terms of both classification accuracy and training speed, sometimes exceeding transductive batch normalization. We recommend that future work in the few-shot / meta-learning community adopt TASKNORM, and if not, declare the form of normalization used and implications thereof, especially where transductive methods are applied.

Acknowledgments

The authors would like to thank Elre Oldewage, Will Tebbutt, and the reviewers for their insightful comments and feedback. Richard E. Turner is supported by Google, Amazon, ARM, Improbable and EPSRC grants EP/M0269571 and EP/L000776/1.

References

- Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G. S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Jozefowicz, R., Kaiser, L., Kudlur, M., Levenberg, J., Mané, D., Monga, R., Moore, S., Murray, D., Olah, C., Schuster, M., Shlens, J., Steiner, B., Sutskever, I., Talwar, K., Tucker, P., Vanhoucke, V., Vasudevan, V., Viégas, F., Vinyals, O., Warden, P., Wattenberg, M., Wicke, M., Yu, Y., and Zheng, X. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. URL <https://www.tensorflow.org/>. Software available from tensorflow.org.
- Ba, J. L., Kiros, J. R., and Hinton, G. E. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- Bakker, B. and Heskes, T. Task clustering and gating for Bayesian multitask learning. *Journal of Machine Learning Research*, 4:83–99, May 2003.
- Chen, Y. A re-implementation of "prototypical networks for few-shot learning". <https://github.com/cyvius96/prototypical-network-pytorch>, 2018.
- Finn, C., Abbeel, P., and Levine, S. Model-agnostic meta-learning for fast adaptation of deep networks. In Precup, D. and Teh, Y. W. (eds.), *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pp. 1126–1135, International Convention Centre, Sydney, Australia, 06–11 Aug 2017. PMLR. URL <http://proceedings.mlr.press/v70/finn17a.html>.
- Finn, C. B. Code for "Model-agnostic meta-learning for fast adaptation of deep networks". <https://github.com/cbfinn/maml>, 2017.
- Gordon, J., Bronskill, J., Bauer, M., Nowozin, S., and Turner, R. Meta-learning probabilistic inference for prediction. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=HkxStoC5F7>.
- Grant, E., Finn, C., Levine, S., Darrell, T., and Griffiths, T. Recasting gradient-based meta-learning as hierarchical bayes. In *International Conference on Learning Representations*, 2018. URL https://openreview.net/forum?id=BJ_UL-k0b.
- Ha, D., Dai, A., and Le, Q. V. Hypernetworks. In *International Conference on Learning Representations*, 2016. URL <https://openreview.net/forum?id=rkpACellx>.
- Heskes, T. Empirical bayes for learning to learn. In *Proceedings of the Seventeenth International Conference on Machine Learning*, ICML '00, pp. 367–374, San Francisco, CA, USA, 2000. Morgan Kaufmann Publishers Inc. ISBN 1-55860-707-2. URL <http://dl.acm.org/citation.cfm?id=645529.658133>.
- Hospedales, T., Antoniou, A., Micaelli, P., and Storkey, A. Meta-learning in neural networks: A survey. *arXiv preprint arXiv:2004.05439*, 2020.
- Ioffe, S. Batch renormalization: Towards reducing minibatch dependence in batch-normalized models. In Guyon, I., Luxburg, U. V., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems 30*, pp. 1945–1953. Curran Associates, Inc., 2017.
- Ioffe, S. and Szegedy, C. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In Bach, F. and Blei, D. (eds.), *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pp. 448–456, Lille, France, 07–09 Jul 2015. PMLR. URL <http://proceedings.mlr.press/v37/ioffe15.html>.
- Jerfel, G., Grant, E., Griffiths, T., and Heller, K. A. Reconciling meta-learning and continual learning with online mixtures of tasks. In Wallach, H., Larochelle, H., Beygelzimer, A., dAlché-Buc, F., Fox, E., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems 32*, pp. 9119–9130. Curran Associates, Inc., 2019.
- Krizhevsky, A. and Hinton, G. Learning multiple layers of features from tiny images. Technical report, Citeseer, 2009.
- Lake, B., Salakhutdinov, R., Gross, J., and Tenenbaum, J. One shot learning of simple visual concepts. In *Proceedings of the annual meeting of the cognitive science society*, volume 33, 2011.
- LeCun, Y., Cortes, C., and Burges, C. MNIST handwritten digit database. *AT&T Labs [Online]*. Available: <http://yann.lecun.com/exdb/mnist>, 2:18, 2010.

- Liu, Y., Lee, J., Park, M., Kim, S., Yang, E., Hwang, S., and Yang, Y. Learning to propagate labels: transductive propagation network for few-shot learning. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=SyVuRiC5K7>.
- Luo, C., Zhan, J., Xue, X., Wang, L., Ren, R., and Yang, Q. Cosine normalization: Using cosine similarity instead of dot product in neural networks. In *International Conference on Artificial Neural Networks*, pp. 382–391. Springer, 2018.
- Mishra, N., Rohaninejad, M., Chen, X., and Abbeel, P. A simple neural attentive meta-learner. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=B1DmUzWAW>.
- Nagabandi, A., Finn, C., and Levine, S. Deep online learning via meta-learning: Continual adaptation for model-based RL. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=HyxAfnA5tm>.
- Nam, H. and Kim, H.-E. Batch-instance normalization for adaptively style-invariant neural networks. In *Advances in Neural Information Processing Systems*, pp. 2558–2567, 2018.
- Nichol, A., Achiam, J., and Schulman, J. On first-order meta-learning algorithms. *arXiv preprint arXiv:1803.02999*, 2018.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. Pytorch: An imperative style, high-performance deep learning library. In Wallach, H., Larochelle, H., Beygelzimer, A., d’Alché-Buc, F., Fox, E., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems 32*, pp. 8024–8035. Curran Associates, Inc., 2019.
- Rajeswaran, A., Finn, C., Kakade, S. M., and Levine, S. Meta-learning with implicit gradients. In Wallach, H., Larochelle, H., Beygelzimer, A., d’Alché-Buc, F., Fox, E., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems 32*, pp. 113–124. Curran Associates, Inc., 2019.
- Ravi, S. and Larochelle, H. Optimization as a model for few-shot learning. In *Proceedings of the International Conference on Learning Representations*, 2017. URL <https://openreview.net/pdf?id=rJY0-Kc1l>.
- Ren, M., Ravi, S., Triantafillou, E., Snell, J., Swersky, K., Tenenbaum, J. B., Larochelle, H., and Zemel, R. S. Meta-learning for semi-supervised few-shot classification. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=HJcSzz-CZ>.
- Requeima, J., Gordon, J., Bronskill, J., Nowozin, S., and Turner, R. E. Fast and flexible multi-task classification using conditional neural adaptive processes. In Wallach, H., Larochelle, H., Beygelzimer, A., d’Alché-Buc, F., Fox, E., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems 32*, pp. 7957–7968. Curran Associates, Inc., 2019a.
- Requeima, J., Gordon, J., Bronskill, J., Nowozin, S., and Turner, R. E. Code for "Fast and flexible multi-task classification using conditional neural adaptive processes". <https://github.com/cambridge-mlg/cnaps>, 2019b.
- Salimans, T. and Kingma, D. P. Weight normalization: A simple reparameterization to accelerate training of deep neural networks. In *Advances in neural information processing systems*, pp. 901–909, 2016.
- Schmidhuber, J. *Evolutionary principles in self-referential learning, or on learning how to learn: the meta-meta-... hook*. PhD thesis, Technische Universität München, 1987.
- Singh, S. and Krishnan, S. Filter response normalization layer: Eliminating batch dependence in the training of deep neural networks. *arXiv preprint arXiv:1911.09737*, 2019.
- Snell, J. Code for the nips 2017 paper "prototypical networks for few-shot learning". <https://github.com/jakesnell/prototypical-networks>, 2017.
- Snell, J., Swersky, K., and Zemel, R. Prototypical networks for few-shot learning. In Guyon, I., Luxburg, U. V., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems 30*, pp. 4077–4087. Curran Associates, Inc., 2017.
- Thrun, S. and Pratt, L. *Learning to learn*. Springer Science & Business Media, 2012.
- Triantafillou, E., Zhu, T., Dumoulin, V., Lamblin, P., Xu, K., Goroshin, R., Gelada, C., Swersky, K., Manzagol, P.-A., and Larochelle, H. Code for "Meta-dataset: A dataset of datasets for learning to learn from few examples". <https://github.com/google-research/meta-dataset>, 2019.

Triantafillou, E., Zhu, T., Dumoulin, V., Lamblin, P., Evci, U., Xu, K., Goroshin, R., Gelada, C., Swersky, K., Manzagol, P.-A., and Larochelle, H. Meta-dataset: A dataset of datasets for learning to learn from few examples. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=rkgAGAVKPr>.

Ulyanov, D., Vedaldi, A., and Lempitsky, V. Instance normalization: The missing ingredient for fast stylization. *arXiv preprint arXiv:1607.08022*, 2016.

Vinyals, O., Blundell, C., Lillicrap, T., kavukcuoglu, k., and Wierstra, D. Matching networks for one shot learning. In Lee, D. D., Sugiyama, M., Luxburg, U. V., Guyon, I., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems 29*, pp. 3630–3638. Curran Associates, Inc., 2016.

Wu, Y. and He, K. Group normalization. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 3–19, September 2018.

A. Additional Normalization Layers

Here we discuss various additional NLs that are relevant to meta-learning.

A.1. Batch Renormalization (BRN)

Batch renormalization (BRN; Ioffe, 2017) is intended to mitigate the issue of non-identically distributed and/or small batches while retaining the training efficiency and stability of CBN. In BRN, the CBN algorithm is augmented with an affine transform with batch-derived parameters which correct for the batch statistics being different from the overall population. The normalized activations of a BRN layer are computed as follows:

$$\mathbf{a}'_n = \gamma \left(r \left(\frac{\mathbf{a}_n - \boldsymbol{\mu}_{BN}}{\boldsymbol{\sigma}_{BN} + \epsilon} \right) + d \right) + \beta,$$

where

$$r = \text{stop_grad} \left(\text{clip}_{[1/r_{max}, r_{max}]} \left(\frac{\boldsymbol{\sigma}_{BN}}{\boldsymbol{\sigma}_r} \right) \right),$$

$$d = \text{stop_grad} \left(\text{clip}_{[-d_{max}, d_{max}]} \left(\frac{\boldsymbol{\mu}_{BN} - \boldsymbol{\mu}_r}{\boldsymbol{\sigma}_r} \right) \right).$$

Here $\text{stop_grad}(\cdot)$ denotes a gradient blocking operation, and $\text{clip}_{[a,b]}$ denotes an operation returning a value in the range $[a, b]$. Like CBN, BRN is not well suited to the meta-learning scenario as it does not map directly to the hierarchical form of meta-learning models. In Section 5, we show that using BRN can improve predictive performance compared to CBN, but still performs significantly worse than competitive approaches. Table 1 shows that batch renormalization performs poorly when using MAML.

A.2. Group Normalization (GN)

A key insight of Wu & He (2018) is that CBN performance suffers with small batch sizes. The goal of Group Normalization (GN; Wu & He, 2018) is thus to address the problem of normalization of small batch sizes, which, among other matters, is crucial for training large models in a data-parallel fashion. This is achieved by dividing the image channels into a number of groups G and subsequently computing the moments for each group. GN is equivalent to LN when there is only a single group ($G = 1$) and equivalent to IN when the number of groups is equal to the number of channels in the layer ($G = C$).

A.3. Other NLs

There exist additional NLs including Weight Normalization (Salimans & Kingma, 2016), Cosine Normalization

(Luo et al., 2018), Filter Response Normalization (Singh & Krishnan, 2019), among many others.

Weight normalization reparameterizes weight vectors in a neural network to improve the conditioning for optimization. Weight normalization is non-transductive, but we don't consider this approach further in this work as we focus on NLs that modify activations as opposed to weights.

Filter Response Normalization (FRN) is another non-transductive NL that performs well for all batch sizes. However we did not include it in our evaluation as FRN also encompasses the activation function as an essential part of normalization making it difficult to be a drop in replacement for CBN in pre-trained networks as is the case for some of our experiments.

Cosine normalization replaces the dot-product calculation in neural networks with cosine similarity for improved performance. We did not consider this method further in our work as it is not a simple drop-in replacement for CBN in pre-existing networks such as the ResNet-18 we use in our experiments.

B. Experimental Details

In this section, we provide the experimental details required to reproduce our experiments. The experiments using MAML (Finn et al., 2017) were implemented in TensorFlow (Abadi et al., 2015), the Prototypical Networks experiments were implemented in Pytorch (Paszke et al., 2019), and the experiments using CNAPs (Requeima et al., 2019a) were implemented using a combination of TensorFlow (Abadi et al., 2015) and Pytorch. All experiments were executed on NVIDIA Tesla P100-16GB GPUs.

B.1. MAML Experiments

We evaluate MAML using a range of normalization layers on:

1. Omniglot (Lake et al., 2011): a few-shot learning dataset consisting of 1623 handwritten characters (each with 20 instances) derived from 50 alphabets.
2. miniImageNet (Vinyals et al., 2016): a dataset of 60,000 color images that is sub-divided into 100 classes, each with 600 instances.

For all the MAML experiments, we used the codebase provided by the MAML authors (Finn, 2017) with only small modifications to enable additional normalization techniques. Note that we used the first-order approximation version of MAML for all experiments. MAML was invoked with the command lines as specified in the `main.py` file in the MAML codebase. No hyper-parameter tuning was performed and we took the results from a single run. All models

were trained for 60,000 iterations and then tested. No early stopping was used. We did not select the model based on validation accuracy or other criteria. The MAML code employs ten gradient steps at test time and computes classification accuracy after each step. We report the maximum accuracy across those ten steps. To generate the plot in Figure 4a, we use the same command line as Omniglot-5-1, but vary the update batch size from one to ten.

B.2. CNAPS Experiments

We evaluate CNAPS using a range of normalization layers on a demanding few-shot classification challenge called Meta-Dataset (Triantafillou et al., 2020). Meta-Dataset is composed of ten (eight train, two test) image classification datasets. We augment Meta-Dataset with three additional held-out datasets: MNIST (LeCun et al., 2010), CIFAR10 (Krizhevsky & Hinton, 2009), and CIFAR100 (Krizhevsky & Hinton, 2009). The challenge constructs few-shot learning tasks by drawing from the following distribution. First, one of the datasets is sampled uniformly; second, the “way” and “shot” are sampled randomly according to a fixed procedure; third, the classes and context / target instances are sampled. Where a hierarchical structure exists in the data (ILSVRC or OMNIGLOT), task-sampling respects the hierarchy. In the meta-test phase, the identity of the original dataset is not revealed and the tasks must be treated independently (i.e. no information can be transferred between them). Notably, the meta-training set comprises a disjoint and dissimilar set of classes from those used for meta-test. Full details are available in Triantafillou et al. (2020).

For all the CNAPS experiments, we use the code provided by the the CNAPS authors (Requeima et al., 2019b) with only small modifications to enable additional normalization techniques. We follow an identical dataset configuration and training process as prescribed in Requeima et al. (2019b). To generate results in Table 2, we used the following CNAPS options: FiLM feature adaptation, a learning rate of 0.001, and TBN, CBN, BRN, and RN used 70,000 training iterations, IN used 200,000 iterations, LN used 110,000 iterations, and TASKNORM used 60,000 iterations. The CNAPS code generates two models: fully trained and best validation. We report the better of the two. We performed no hyper-parameter tuning and report the test results from the first run. Note that CBN, TBN, and RN share the same trained model. They differ only in how meta-testing is done.

B.3. Prototypical Networks Experiments

We evaluate the Prototypical Networks (Snell et al., 2017) algorithm with a range of NLs using the same Omniglot, miniImageNet, and Meta-Dataset benchmarks.

For Omniglot, we used the codebase created by the Prototypical Networks authors (Snell, 2017). For miniIma-

geNet, we used the a different codebase ((Chen, 2018)) as the first codebase did not support miniImageNet. Only small modifications were made to the two codebases to enable additional NLs. For Omniglot and miniImageNet, we set hyper-parameters as prescribed in (Snell et al., 2017). Early stopping was employed and the model that produced the best validation was used for testing.

For Meta-Dataset, we use the code provided by the the CNAPS authors (Requeima et al., 2019b) with only small modifications to enable additional normalization techniques and a new classifier adaptation layer to generate the linear classifier weights per equation (8) in (Snell et al., 2017). We follow an identical dataset configuration and training process as prescribed in Requeima et al. (2019b). To generate results in Table 3, we used the following CNAPS options: no feature adaptation, a learning rate of 0.001, 60,000 training iterations for all NLs, and the pretrained feature extractor weights were not frozen and allowed to update during meta-training.

C. Additional Classification Results

Table C.1 shows the classification accuracy results for the ProtoNets algorithm on the Omniglot and miniImageNet datasets. Figure C.1a and Figure C.1b show the training curves for the ProtoNets algorithm on Omniglot and Meta-Dataset, respectively.

D. Additional Transduction Tests

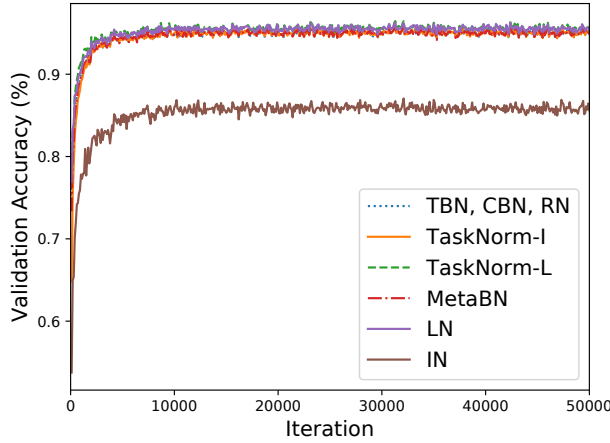
A non-transductive meta-learning system makes predictions for a single test set label conditioned only on a single input and the context set. A transductive meta-learning system also conditions on additional samples from the test set.

Table D.2 demonstrates failure modes for transductive learning. In addition to reporting the classification accuracy results when the target set is evaluated all at once (first column of results for each NL), we report the classification accuracy when meta-testing is performed one target-set *example* at a time (second column of results for each NL), and one target-set *class* at a time (third column of results for each NL). Table D.2 demonstrates that classification accuracy drops dramatically for TBN when testing is performed one example or one class at a time.

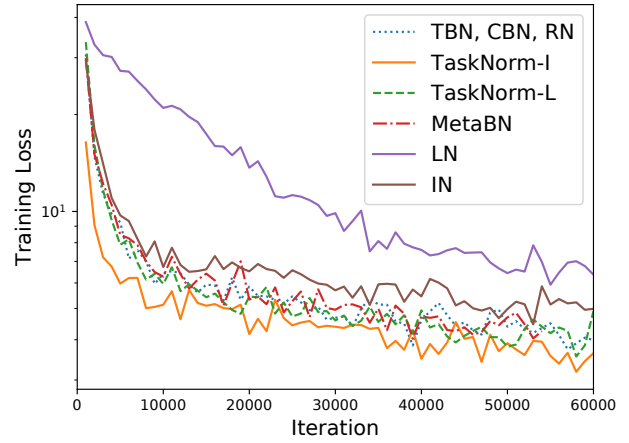
Importantly, in the case of TASKNORM-I (or any non-transductive NL – i.e. all of NLs evaluated in this work apart from TBN), the evaluation results are identical whether they are meta-tested on the entire target set at once, one example at a time, or one class at a time. This shows that transductive learning is sensitive to the distribution over the target set used during meta-training, demonstrating that transductive learning is less generally applicable than non-transductive

Table C.1. Accuracy results for different few-shot settings on Omniglot and miniImageNet using the Prototypical Networks algorithm. All figures are percentages and the $\pm$ sign indicates the 95% confidence interval. Bold indicates the highest scores. The numbers after the configuration name indicate the way and shots, respectively. The vertical lines in the TBN column indicate that this method is transductive.

Configuration	TBN	CBN	BRN	LN	IN	RN	MetaBN	TaskNorm-L	TaskNorm-I
Omniglot-5-1	98.4 $\pm$ 0.2	98.5$\pm$0.2	98.5$\pm$0.2	98.7$\pm$0.2	93.7 $\pm$ 0.4	98.0 $\pm$ 0.2	98.4 $\pm$ 0.2	98.6$\pm$0.2	98.4 $\pm$ 0.2
Omniglot-5-5	99.6$\pm$0.1	99.6$\pm$0.1	99.6$\pm$0.1	99.7$\pm$0.1	98.8 $\pm$ 0.1	99.6$\pm$0.1	99.6$\pm$0.1	99.6$\pm$0.1	99.6$\pm$0.1
Omniglot-20-1	94.5 $\pm$ 0.2	94.5 $\pm$ 0.2	94.6 $\pm$ 0.2	94.9$\pm$0.2	83.5 $\pm$ 0.3	94.1 $\pm$ 0.2	94.5 $\pm$ 0.2	95.0$\pm$0.2	93.4 $\pm$ 0.2
Omniglot-20-5	98.6$\pm$0.1	98.6$\pm$0.1	98.6$\pm$0.1	98.7$\pm$0.1	96.3 $\pm$ 0.1	98.6$\pm$0.1	98.6$\pm$0.1	98.7$\pm$0.1	98.6$\pm$0.1
miniImageNet-5-1	45.9 $\pm$ 0.6	47.8$\pm$0.6	46.3 $\pm$ 0.6	47.5$\pm$0.6	30.4 $\pm$ 0.5	39.7 $\pm$ 0.5	42.6 $\pm$ 0.6	47.5$\pm$0.6	43.2 $\pm$ 0.6
miniImageNet-5-5	65.5 $\pm$ 0.5	66.7$\pm$0.5	64.7 $\pm$ 0.5	66.3$\pm$0.5	48.8 $\pm$ 0.5	63.1 $\pm$ 0.5	64.6 $\pm$ 0.5	65.3 $\pm$ 0.5	63.9 $\pm$ 0.5
Average Rank	4.58	3.25	4.33	2.75	9.00	6.67	5.25	3.08	6.08



(a)



(b)

Figure C.1. (a) Plot of validation accuracy versus training iteration using ProtoNets for Omniglot 20-way, 1-shot corresponding to the results in Table C.1. (b) Training Loss versus iteration corresponding to the results using the ProtoNets algorithm on META-DATASET in Table 3. Note that TBN, CBN, and RN all share the same meta-training step.

learning. In particular, transductive learners may fail to make good predictions if target sets contains a different class balance than what was observed during meta-training, or if they are required to make predictions for one example at a time (e.g. in streaming applications).

E. Ablation Study: Choosing the best parameterization for α

There are a number of possibilities for the parameterization of the TASKNORM blending parameter α . We consider four different configurations for each NL:

1. α is learned separately for each channel (i.e. channel specific) as an independent parameter.
2. α is learned shared across all channels as an independent parameter.
3. α is learned separately for each channel (i.e. channel

specific) as a function of context set size (i.e. $\alpha = \text{SIGMOID}(\text{SCALE}|D^\tau| + \text{OFFSET})$).

4. α is learned shared across all channels as a function of context set size (i.e. $\alpha = \text{SIGMOID}(\text{SCALE}|D^\tau| + \text{OFFSET})$).

Accuracy Table E.3 and Table E.4 show classification accuracy for the various parameterizations for MAML and the CNAPS algorithms, respectively using the TASKNORM-I NL.

When using the MAML algorithm, there are only two options to evaluate as the context size is fixed for each configuration of dataset, shot, and way and thus we need only evaluate the independent options (1 and 2 above). Table E.3 indicates that the classification accuracy for the channel specific and shared parameterizations are nearly identical, but the shared parameterization is better in the Omniglot-5-1 benchmark and hence has the best ranking overall.

Table D.2. Few-shot classification results for TBN and TASKNORM-I on META-DATASET using the CNAPS algorithm. For each NL, the first column of results "All" reports accuracy when meta-testing is performed on the entire target set at once. The second column of results "Example" reports accuracy when meta-testing is performed one example at a time. The third column of results "Class" reports accuracy when meta-testing is performed one class at a time. All figures are percentages and the $\pm$ sign indicates the 95% confidence interval over tasks. Meta-training is performed on datasets above the dashed line, while datasets below the dashed line are entirely held out.

Dataset	TBN			TASKNORM-I		
	All	Example	Class	All	Example	Class
ILSVRC	50.2 $\pm$ 1.0	9.5 $\pm$ 0.3	11.8 $\pm$ 0.4	50.4 $\pm$ 1.1	50.4 $\pm$ 1.1	50.4 $\pm$ 1.1
Omniglot	91.4 $\pm$ 0.5	7.5 $\pm$ 0.4	9.6 $\pm$ 0.4	91.3 $\pm$ 0.6	91.3 $\pm$ 0.6	91.3 $\pm$ 0.6
Aircraft	81.6 $\pm$ 0.6	11.8 $\pm$ 0.4	14.4 $\pm$ 0.4	83.8 $\pm$ 0.6	83.8 $\pm$ 0.6	83.8 $\pm$ 0.6
Birds	74.5 $\pm$ 0.8	7.6 $\pm$ 0.4	8.4 $\pm$ 0.4	74.4 $\pm$ 0.9	74.4 $\pm$ 0.9	74.4 $\pm$ 0.9
Textures	59.7 $\pm$ 0.7	17.0 $\pm$ 0.2	18.1 $\pm$ 0.4	61.1 $\pm$ 0.7	61.1 $\pm$ 0.7	61.1 $\pm$ 0.7
Quick Draw	70.8 $\pm$ 0.8	5.6 $\pm$ 0.4	8.8 $\pm$ 0.4	74.7 $\pm$ 0.7	74.7 $\pm$ 0.7	74.7 $\pm$ 0.7
Fungi	46.0 $\pm$ 1.0	5.0 $\pm$ 0.3	6.5 $\pm$ 0.4	50.6 $\pm$ 1.1	50.6 $\pm$ 1.1	50.6 $\pm$ 1.1
VGG Flower	86.6 $\pm$ 0.5	11.2 $\pm$ 0.4	12.6 $\pm$ 0.4	87.8 $\pm$ 0.5	87.8 $\pm$ 0.5	87.8 $\pm$ 0.5
Traffic Signs	66.6 $\pm$ 0.9	6.0 $\pm$ 0.3	8.1 $\pm$ 0.4	64.8 $\pm$ 0.8	64.8 $\pm$ 0.8	64.8 $\pm$ 0.8
MSCOCO	41.3 $\pm$ 1.0	6.1 $\pm$ 0.3	7.9 $\pm$ 0.4	42.2 $\pm$ 1.0	42.2 $\pm$ 1.0	42.2 $\pm$ 1.0
MNIST	92.1 $\pm$ 0.4	14.4 $\pm$ 0.3	19.3 $\pm$ 0.4	91.3 $\pm$ 0.4	91.3 $\pm$ 0.4	91.3 $\pm$ 0.4
CIFAR10	70.1 $\pm$ 0.8	14.4 $\pm$ 0.3	16.4 $\pm$ 0.4	70.0 $\pm$ 0.8	70.0 $\pm$ 0.8	70.0 $\pm$ 0.8
CIFAR100	55.6 $\pm$ 1.0	5.6 $\pm$ 0.3	7.7 $\pm$ 0.4	54.6 $\pm$ 1.0	54.6 $\pm$ 1.0	54.6 $\pm$ 1.0

Table E.3. Few-shot classification results for two α parameterizations on Omniglot and miniImageNet using the MAML algorithm. All figures are percentages and the $\pm$ sign indicates the 95% confidence interval over tasks. Bold indicates the highest scores.

Configuration	Independent	
	Channel Specific	Shared
Omniglot-5-1	90.7 $\pm$ 1.0	94.4$\pm$0.8
Omniglot-5-5	98.3 $\pm$ 0.2	98.6$\pm$0.2
Omniglot-20-1	90.6$\pm$0.5	90.0$\pm$0.5
Omniglot-20-5	96.4$\pm$0.2	96.3$\pm$0.2
miniImageNet-5-1	42.6$\pm$1.8	42.4$\pm$1.7
miniImageNet-5-5	58.8$\pm$0.9	58.7$\pm$0.9
Average Rank	1.67	1.33

When using the CNAPS algorithm on the Meta-Dataset benchmark, the best parameterization option in terms of classification accuracy is α shared across channels as a function of context size. One justification for having α be a function of context size can be seen in Figure 3b. Here we plot the line $\text{SCALE}|D^\tau| + \text{OFFSET}$ on a linear scale for a representative set of NLs in the ResNet-18 used in the CNAPS algorithm. The algorithm has learned that the SCALE parameter is non-zero and the OFFSET is almost zero in all cases. If a constant α would lead to better accuracy, we would see the opposite (i.e the SCALE parameter would be at or near zero and the OFFSET parameter being some non-zero value). From Table E.4 we can also see that accuracy is better when the parameterization is a shared α opposed to having a channel-specific α .

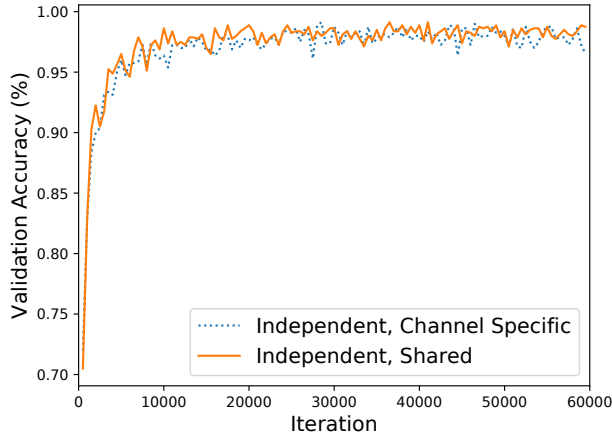
Training Speed Figure E.2a and Figure E.2b show the learning curves for the various parameterization options using the MAML and the CNAPS algorithms, respectively with a TASKNORM-I NL.

For the MAML algorithm the training efficiency of the shared and channel specific parameterizations are almost identical. For the CNAPS algorithm, Figure E.2b indicates the training efficiency of the independent parameterization is considerably worse than the functional one. The two functional representations for the CNAPS algorithm have almost identical training curves. Based on Figure E.2a and Figure E.2b, we conclude that the training speed of the functional parameterization is superior to that of the independent parameterization and that there is little or no difference in the training speeds between the functional, shared parameterization and the functional, channel specific parameterization.

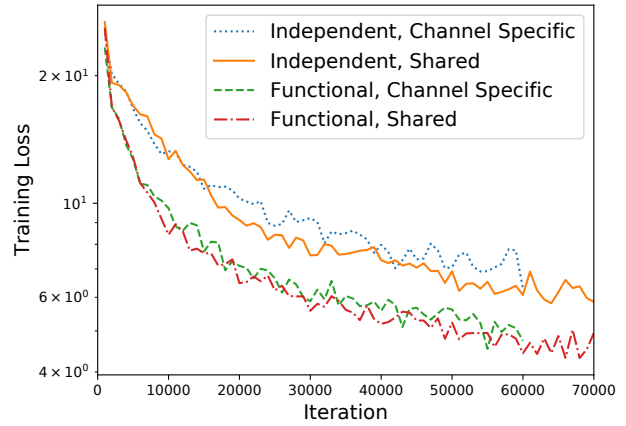
In summary, the best parameterization for α when it is learned shared across channels as a function of context set size (option 4, above). We use this parameterization in all of the CNAPS experiments in the main paper. For the MAML experiments, the functional parameterization is meaningless given that all the test configurations have a fixed context size. In that case, we used the independent, shared across channels parameterization for α for the experiments in the main paper.

Table E.4. Few-shot classification results for various α parameterizations on META-DATASET using the CNAPS algorithm. All figures are percentages and the $\pm$ sign indicates the 95% confidence interval over tasks. Bold indicates the highest scores. Meta-training performed on datasets above the dashed line, while datasets below the dashed line are entirely held out.

Dataset	Independent		Functional	
	Channel Specific	Shared	Channel Specific	Shared
ILSVRC	45.3 $\pm$ 1.0	49.6$\pm$1.1	49.8$\pm$1.1	50.6$\pm$1.1
Omniglot	90.8$\pm$0.6	90.9$\pm$0.6	90.1$\pm$0.6	90.7$\pm$0.6
Aircraft	82.3 $\pm$ 0.7	84.6$\pm$0.6	84.4$\pm$0.6	83.8$\pm$0.6
Birds	70.1 $\pm$ 0.9	73.2 $\pm$ 0.9	73.1 $\pm$ 0.9	74.6$\pm$0.8
Textures	54.8 $\pm$ 0.7	58.5 $\pm$ 0.7	61.0 $\pm$ 0.8	62.1$\pm$0.7
Quick Draw	73.0 $\pm$ 0.8	73.9$\pm$0.7	74.2$\pm$0.7	74.8$\pm$0.7
Fungi	43.8 $\pm$ 1.0	47.6$\pm$1.0	48.0$\pm$1.0	48.7$\pm$1.0
VGG Flower	85.9 $\pm$ 0.6	86.3 $\pm$ 0.5	86.5 $\pm$ 0.7	89.6$\pm$0.6
Traffic Signs	62.6 $\pm$ 0.8	62.6 $\pm$ 0.8	60.1 $\pm$ 0.8	67.0$\pm$0.7
MSCOCO	38.3 $\pm$ 1.1	40.9 $\pm$ 1.0	40.2 $\pm$ 1.0	43.4$\pm$1.0
MNIST	92.6$\pm$0.4	91.7 $\pm$ 0.4	91.1 $\pm$ 0.4	92.3$\pm$0.4
CIFAR10	65.7 $\pm$ 0.9	67.7 $\pm$ 0.8	67.3 $\pm$ 0.9	69.3$\pm$0.8
CIFAR100	48.1 $\pm$ 1.2	52.1 $\pm$ 1.1	53.3$\pm$1.0	54.6$\pm$1.1
Average Rank	3.5	2.5	2.5	1.5



(a)



(b)

Figure E.2. (a) Plots of validation accuracy versus training iteration corresponding to the parameterization experiments using the MAML algorithm in Table E.3. (b) Plot of training loss versus iteration corresponding to the parameterization experiments using the CNAPS algorithm in Table E.4.